BHOOMI ANNAMDEVARA
24BCI0130

COMPILER DESIGN LAB
Lab Assignment - 02
Generate Lexical Analyzer using LEX Program

QUESTION 1:
A data entry system needs to track the number of alphabets and digits entered by a user in a form field for statistical analysis. Write a Lex program that reads an input string and counts number of alphabets and digits.

Aim:
To write a Lex program that reads an input string and counts the number of alphabets and digits present in it.

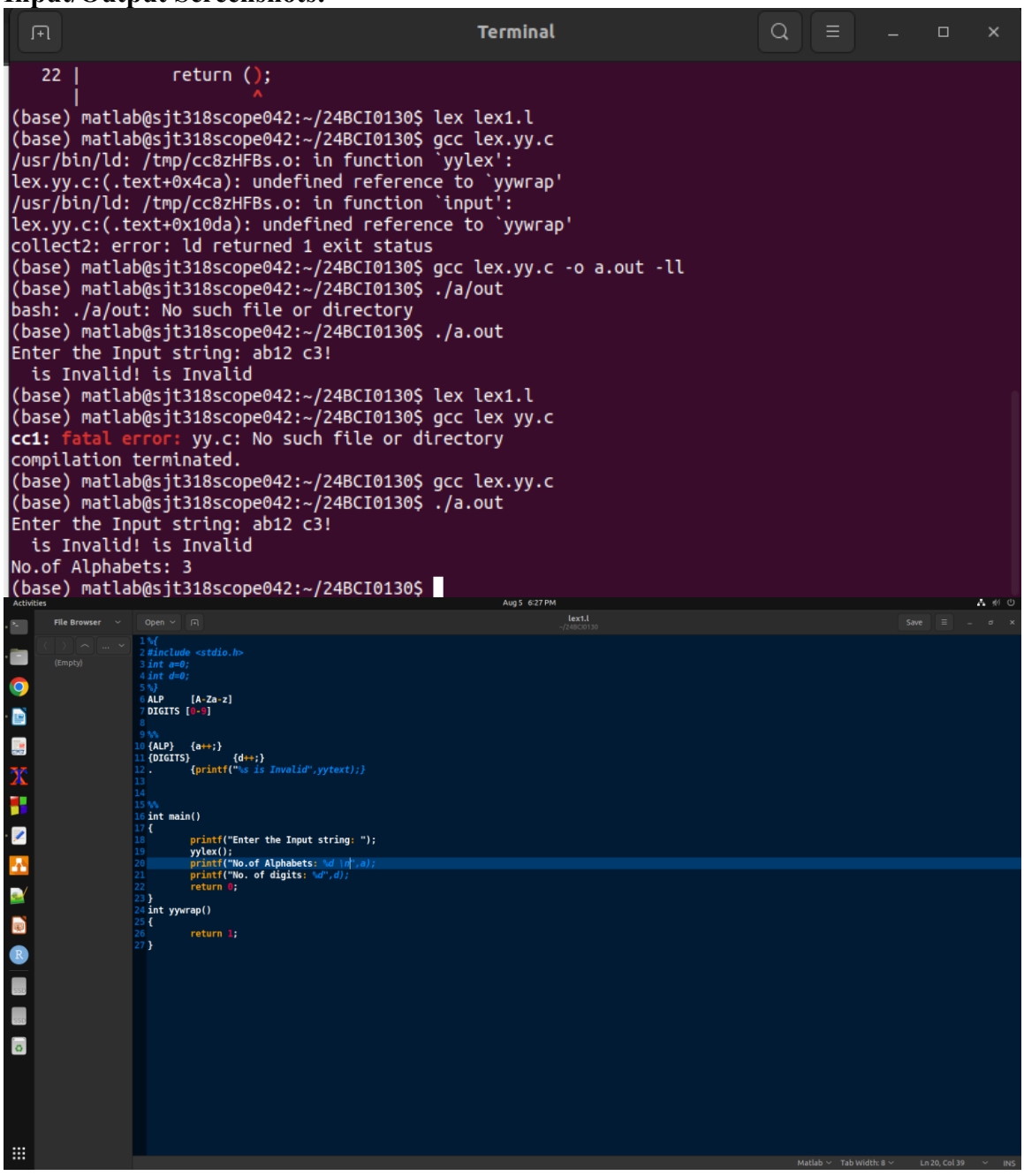
Algorithm
Start.
Initialize two counters: a = 0 and d = 0.
Read the input string.
If the character is an alphabet (A-Z or a-z), increment a.
If the character is a digit (0-9), increment d.
If the character matches neither pattern, display it as invalid.
Display the total number of alphabets and digits.
Stop.

Program:

```
%{
#include <stdio.h>
int a=0;
int d=0;
%}
ALP     [A-Za-z]
DIGITS  [0-9]
%%
{ALP}       {a++;}
{DIGITS}    {d++;}
.           {printf("%s is Invalid",yytext);}
%%
int main()
{
        printf("Enter the Input string: ");
        yylex();
        printf("No.of Alphabets: %d\n",a);
        printf("No. of digits: %d\n",d);
        return 0;
}
int yywrap()
{
```

```
        return 1;
}
```

Input/Output Screenshots:



Result

Thus, the Lex program to count the number of alphabets and digits was implemented and executed successfully.

QUESTION 2:
A typing tutor software is designed to monitor a student's typing performance during practice sessions. As part of the evaluation, it needs to keep track of how many vowels and consonants the student has typed. Write a Lex program that reads an input text and counts the number of vowels (both uppercase and lowercase) and the number of consonants (both uppercase and lowercase).

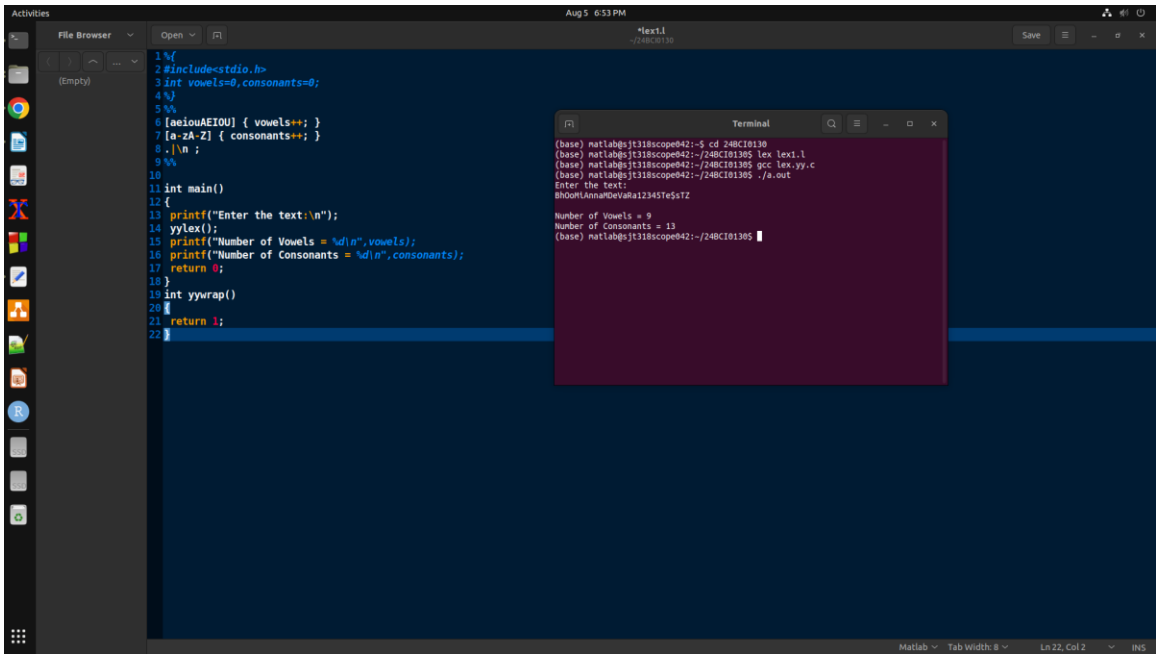
Aim:
To write a Lex program that reads an input text and counts the number of vowels and consonants present in it.

Algorithm
Start.
Initialize two counters: vowels = 0 and consonants = 0.
Read the input text character by character.
If the character is a vowel (a, e, i, o, u - upper or lower case), increment vowels.
Else if the character is an alphabet, increment consonants.
Ignore all other characters.
Display the total number of vowels and consonants.
Stop.

Program:

```
%{
#include<stdio.h>
int vowels=0,consonants=0;
%}
%%
[aeiouAEIOU]     { vowels++; }
[a-zA-Z]         { consonants++; }
.|\n            ;
%%
int main()
{
  printf("Enter the text:\n");
  yylex();
  printf("Number of Vowels = %d\n",vowels);
  printf("Number of Consonants = %d\n",consonants);
  return 0;
}
int yywrap()
{
  return 1;
}
```

Input/Output Screenshots:



Result

Thus, the Lex program to count the number of vowels and consonants was implemented and executed successfully.

QUESTION 3:
A text analysis tool needs to provide quick statistics about any given document or text input. As part of its functionality, it should count the total number of lines, words, and characters entered by the user. Write a Lex program that reads an input stream and counts the number of lines, words and characters.

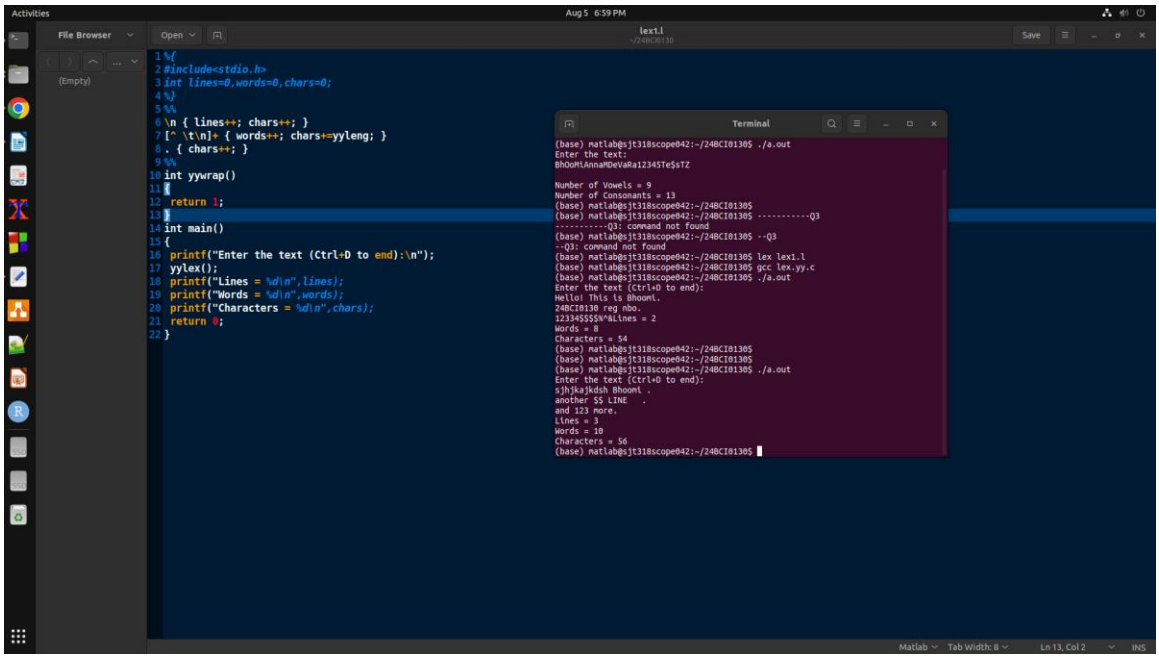
Aim:
To write a Lex program that reads an input stream and counts the number of lines, words and characters present in it.

Algorithm
Start.
Initialize counters: lines = 0, words = 0, chars = 0.
Read the input stream.
On encountering a newline character, increment lines and chars.
On encountering a sequence of non-whitespace characters, increment words and add its length to chars.
For every other character, increment chars.
Display the total number of lines, words and characters.
Stop.

Program:

```
%{
#include<stdio.h>
int lines=0,words=0,chars=0;
%}
%%
\n           { lines++; chars++; }
[^ \t\n]+   { words++; chars+=yyleng; }
.            { chars++; }
%%
int yywrap()
{
  return 1;
}
int main()
{
  printf("Enter the text (Ctrl+D to end):\n");
  yylex();
  printf("Lines = %d\n",lines);
  printf("Words = %d\n",words);
  printf("Characters = %d\n",chars);
  return 0;
}
```

Input/Output Screenshots:



Result

Thus, the Lex program to count the number of lines, words and characters was implemented and executed successfully.

QUESTION 4:
In the lexical analysis phase of a compiler's front-end, the system must verify whether an input string is a valid variable name (identifier) or a valid number (constant) according to programming language rules. Write a Lex program that reads an input string and determines whether it is a valid identifier (starting with a letter or underscore or dollar sign, followed by letters, digits, or underscores) or a valid number (consisting only of digits).

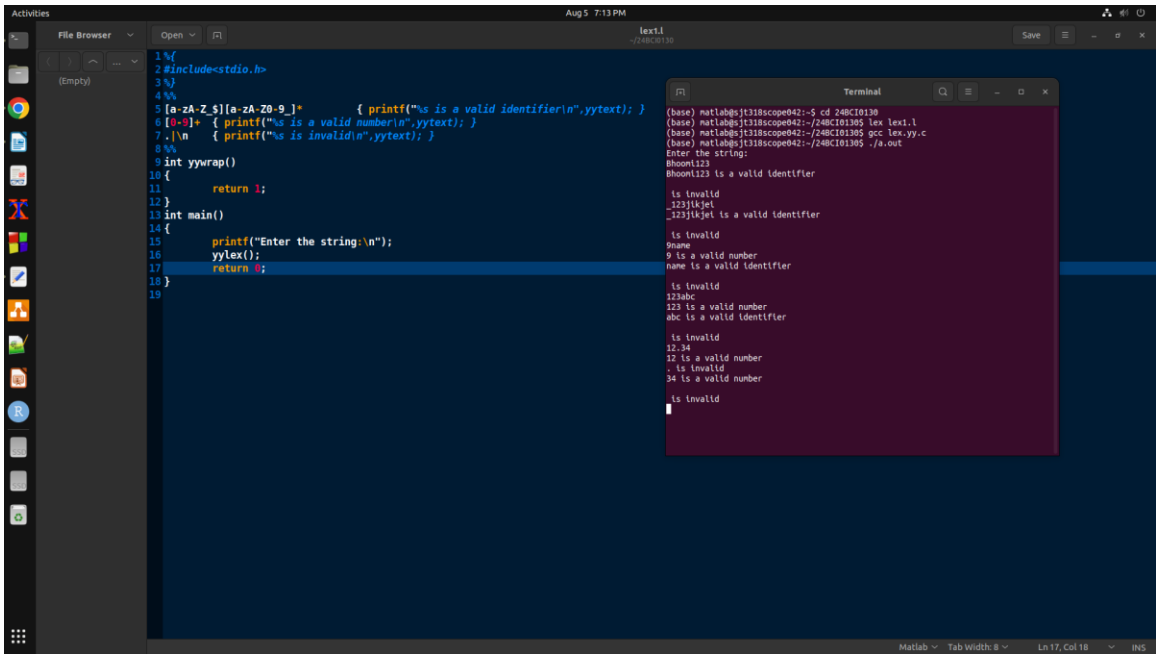
Aim:
To write a Lex program that reads an input string and determines whether it is a valid identifier or a valid number.

Algorithm
Start.
Read the input string.
If the string starts with a letter, underscore, or dollar sign, followed by letters, digits, or underscores, classify it as a valid identifier.
If the string consists only of digits, classify it as a valid number.
If neither pattern matches, classify the string as invalid.
Display the result.
Stop.

Program:

```
%{
#include<stdio.h>
%}
%%
[a-zA-Z_$][a-zA-Z0-9_]*   { printf("%s is a valid
identifier\n",yytext); }
[0-9]+                     { printf("%s is a valid number\n",yytext); }
.|\n                      { printf("%s is invalid\n",yytext); }
%%
int yywrap()
{
        return 1;
}
int main()
{
  printf("Enter the string:\n");
  yylex();
  return 0;
}
```

Input/Output Screenshots:



Result

Thus, the Lex program to validate an identifier or a number was implemented and executed successfully.

QUESTION 5:
A calculator application's backend needs to process user input and determine whether the entered numeric value is an integer or a real (decimal) number before performing calculations. Write a Lex program that reads a numeric input and classifies it as: Integer and Real Number.

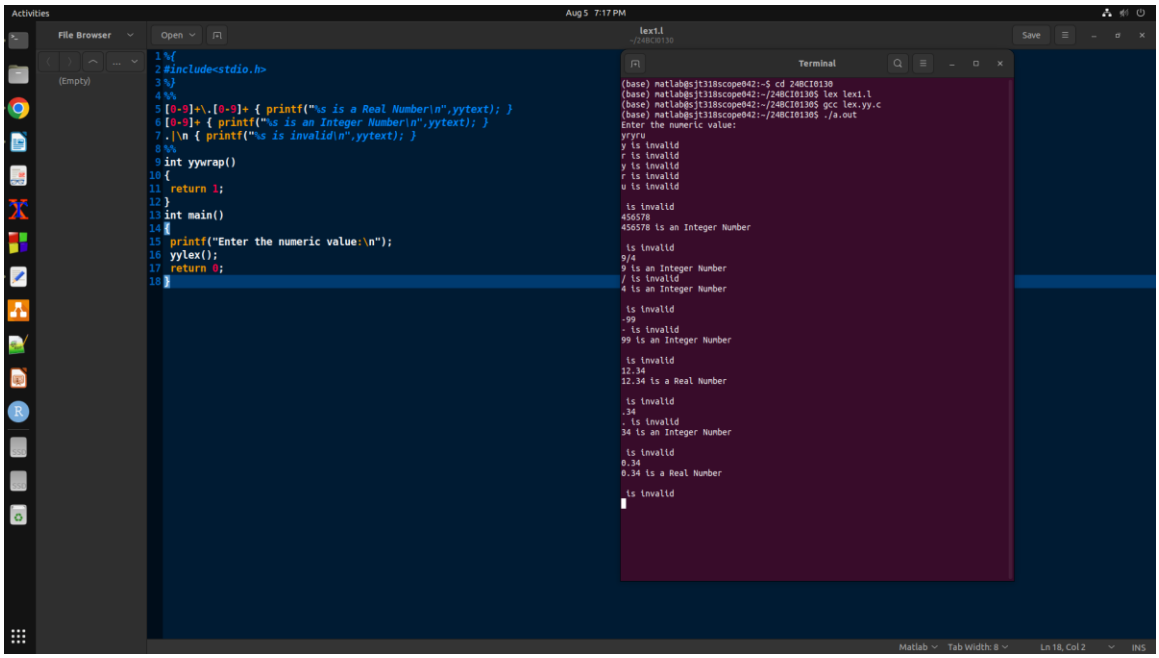
Aim:
To write a Lex program that reads a numeric input and classifies it as an integer or a real number.

Algorithm
Start.
Read the numeric input.
If the input consists of digits followed by a decimal point and more digits, classify it as a Real Number.
If the input consists only of digits, classify it as an Integer Number.
If neither pattern matches, classify the input as invalid.
Display the result.
Stop.

Program:

```
%{
#include<stdio.h>
%}
%%
[0-9]+\.[0-9]+   { printf("%s is a Real Number\n",yytext); }
[0-9]+            { printf("%s is an Integer Number\n",yytext); }
.|\n              { printf("%s is invalid\n",yytext); }
%%
int yywrap()
{
  return 1;
}
int main()
{
  printf("Enter the numeric value:\n");
  yylex();
  return 0;
}
```

Input/Output Screenshots:



Result

Thus, the Lex program to classify a numeric input as an integer or a real number was implemented and executed successfully.

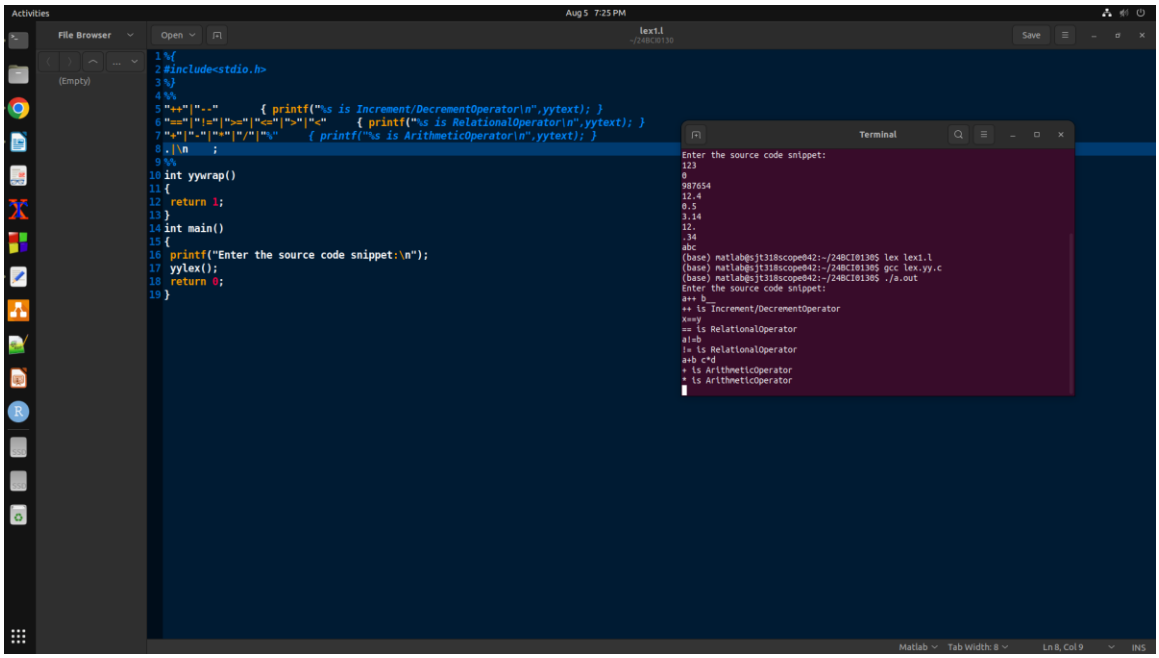
QUESTION 6:
A code highlighter tool is used in an Integrated Development Environment (IDE) to enhance the readability of source code. As part of its functionality, it must detect and classify different types of operators present in the code. Write a Lex program that reads a source code snippet and detects the following categories of operators: Arithmetic Operators (+, -, *, /, %), Relational Operators (==, !=, >, <, >=, <=), Increment and Decrement Operators (++, --).

Aim:
To write a Lex program that reads a source code snippet and detects and classifies arithmetic, relational, and increment/decrement operators.

Algorithm
Start.
Read the source code snippet.
If the token matches ++ or --, classify it as an Increment/Decrement Operator.
If the token matches ==, !=, >=, <=, > or <, classify it as a Relational Operator.
If the token matches +, -, *, / or %, classify it as an Arithmetic Operator.
Ignore all other characters.
Display the detected operators along with their category.
Stop.

Program:
```
%{
#include<stdio.h>
%}
%%
"++"|"--"                    { printf("%s is
Increment/DecrementOperator\n",yytext); }
"=="|"!="|">="|"<="|">"|"<"   { printf("%s is
RelationalOperator\n",yytext); }
"+"|"-"|"*"|"/"|"%"           { printf("%s is
ArithmeticOperator\n",yytext); }
.|\n                         ;
%%
int yywrap()
{
  return 1;
}
int main()
{
  printf("Enter the source code snippet:\n");
  yylex();
  return 0;
}
```

Input/Output Screenshots:



Result

Thus, the Lex program to detect and classify arithmetic, relational, and increment/decrement operators was implemented and executed successfully.

QUESTION 7:
Write a Lex program that validates user input for an online KYC form by checking whether the entered details match the following specified formats: Aadhar Number (12-digit numeric format), Mobile Number (10-digit numeric format starting with digits 6-9), PAN Number (Format ABCDE1234F: 5 uppercase letters, 4 digits, 1 uppercase letter), Voter ID (Format ABC1234567: 3 uppercase letters followed by 7 digits), VIT Student Register Number, and VIT Student/Faculty Email ID in their respective valid formats. The program should read each input string and display whether it matches one of the specified valid formats.

Aim:
To write a Lex program that validates KYC form details such as Aadhar Number, Mobile Number, PAN Number, Voter ID, VIT Register Number, and VIT Student/Faculty Email ID.

Algorithm
Start.
Read the input line by line.
If the line is 12 digits, classify it as a valid Aadhar Number.
If the line is 10 digits starting with 6-9, classify it as a valid Mobile Number.
If the line matches 5 uppercase letters, 4 digits and 1 uppercase letter, classify it as a valid PAN Number.
If the line matches 3 uppercase letters followed by 7 digits, classify it as a valid Voter ID.
If the line matches 2 digits, 3 uppercase letters and 4 digits, classify it as a valid VIT Register Number.
If the line ends with @vitstudent.ac.in or @vit.ac.in, classify it as a valid VIT Student or Faculty Email ID respectively.
If none of the formats match, classify the input as invalid.
Display the result for each input.
Stop.

Program:
```
%{
#include<stdio.h>
%}
%%
^[0-9]{12}$                          { printf("%s -> Valid Aadhar
Number\n",yytext); }
^[6-9][0-9]{9}$                      { printf("%s -> Valid Mobile
Number\n",yytext); }
^[A-Z]{5}[0-9]{4}[A-Z]$              { printf("%s -> Valid PAN
Number\n",yytext); }
^[A-Z]{3}[0-9]{7}$                   { printf("%s -> Valid Voter
ID\n",yytext); }
^[0-9]{2}[A-Z]{3}[0-9]{4}$           { printf("%s -> Valid VIT
Register Number\n",yytext); }
^[a-zA-Z0-9._]+@vitstudent\.ac\.in$   { printf("%s -> Valid VIT Student
Email ID\n",yytext); }
```

```
^[a-zA-Z0-9._]+@vit\.ac\.in$          { printf("%s -> Valid VIT Faculty
Email ID\n",yytext); }
^.+$                                   { printf("%s -> Invalid
Format\n",yytext); }
\n                                     ;
%%
int yywrap()
{
    return 1;
}
int main()
{
    printf("Enter the details line by line (Ctrl+D to end):\n");
    yylex();
    return 0;
}
```

Input/Output Screenshots:
(not completed as per instructions to be left)

Result
Thus, the Lex program to validate KYC form details against the specified formats was implemented and executed successfully.

QUESTION 8:
In the development of a mini compiler, the lexical analysis phase is responsible for scanning the source code and breaking it down into meaningful tokens for further processing. These tokens need to be categorized for syntax analysis. Write a Lex program that takes a C program as input and performs lexical analysis to categorize each token into one of the following types: Keywords, Identifiers, Numbers, Operators, Special Symbols, Strings. The program should read the C source code, identify each token, and display it along with its corresponding category.

Aim:
To write a Lex program that reads a C source code and categorizes each token as a keyword, identifier, number, operator, special symbol, or string.

Algorithm
Start.
Read the C source code.
If the token is enclosed in double quotes, classify it as a String.
If the token is a numeric value, classify it as a Number.
If the token matches a predefined keyword list (int, float, if, else, while, etc.), classify it as a Keyword.
If the token is an alphanumeric identifier not in the keyword list, classify it as an Identifier.
If the token matches an operator symbol, classify it as an Operator.
If the token matches a special symbol such as brackets, braces, or semicolons, classify it as a Special Symbol.
Display each token along with its category.
Stop.

Program:
```
%{
#include<stdio.h>
#include<string.h>
char
*keywords[]={"int","float","char","double","if","else","while","for",
                    "return","void","break","continue","switch","case",
                    "default","do","struct","typedef","const","static"};
int is_keyword(char *s)
{
    int i;
    for(i=0;i<20;i++)
        if(strcmp(keywords[i],s)==0) return 1;
    return 0;
}
%}
%%
\"[^\"]*\"                { printf("%-15s -> String\n",yytext); }
[0-9]+(\.[0-9]+)?           { printf("%-15s -> Number\n",yytext); }
```

```
[a-zA-Z_][a-zA-Z0-9_]*       { if(is_keyword(yytext)) printf("%-15s ->
Keyword\n",yytext);
                               else printf("%-15s ->
Identifier\n",yytext); }
"=="|"!="|"<="|">="|"&&"|"||"|"++"|"--"|"+"|"-
"|"*"|"/"|"%"|"="|"<"|">"|"!"
                              { printf("%-15s -> Operator\n",yytext); }
[(){}\[\];,]                 { printf("%-15s -> Special
Symbol\n",yytext); }
[ \t\n]+                     ;
.                             ;
%%
int yywrap()
{
    return 1;
}
int main()
{
    printf("Enter the C program (Ctrl+D to end):\n");
    yylex();
    return 0;
}
```

Input/Output Screenshots:

(not completed as per instructions to be left)

Result

Thus, the Lex program to perform lexical analysis and categorize tokens of a C program was implemented and executed successfully.